BÀI TẬP CÁ NHÂN 3 — VT-062

Sinh viên: Đào Ngọc Cường · **MSSV:** 23001841 · **Team ID:** SE2026-T16
Vai trò: Nhóm trưởng; phần việc trước đây chủ yếu TV2/backend
Dự án: BC-03 — DAO Budget Simulator
Repository chính thức: https://github.com/SE2026-21-1/dao-budget-simulator
Baseline: ae8c3533e14a01f74ba4ea7b00c74365e1651be1
Nhánh cá nhân local: individual/23001841-vt062-idempotent-retry (chưa publish)

> **Cam kết theo đề:** "Tôi chịu trách nhiệm về toàn bộ code, test và nội dung bài làm; tôi đã khai báo việc sử dụng AI."
> Dòng cam kết được đưa vào báo cáo theo yêu cầu bài tập; đây **không** phải chữ ký hay xác nhận sinh viên đã review/chấp thuận patch. Reflection ở cuối là bản nháp AI hỗ trợ để sinh viên tự chỉnh.

1. Vấn đề, phạm vi và contract

Problem statement. Khi ví đã trả hash H1 cho yêu cầu tạo proposal nhưng RPC chờ receipt lỗi tạm thời, người dùng bấm lại cùng bản nháp có thể khiến frontend gọi ví gửi thêm giao dịch. Source Governor tạo proposal ID từ exact payload; với payload hoàn toàn giống nhau giao dịch H2 dự kiến bị từ chối, nhưng H2 vẫn là một side effect thừa. Finding F1 còn cho thấy: ngay cả khi receipt H1 thành công, bước đọc ProposalCreated không trả Proposal ID thì form từng cho gửi thêm. VT-062 sửa lớp ứng dụng trong cùng phiên trang; không dùng chống ghi trùng event của indexer hoặc chống giải ngân trùng ở Treasury để thay bằng chứng này.

Acceptance criteria kiểm thử. AC1: sau khi có H1, lỗi chờ receipt tạm thời giữ H1 và draft/requestId; chưa báo success. AC2: retry cùng identity sau lỗi tạm chỉ tiếp tục chờ H1, không gọi ví lần hai. AC3: receipt H1 thành công được ghi nhận đúng một lần; có ID thì hiển thị ID, thiếu/không đọc được ID vẫn hiển thị success/H1 và chặn gửi lại ở handler. AC4: lỗi trước hash cho phép thử ký lại; revert/cancel là terminal, không bị coi là timeout. AC5: payload/account/chain/deployment khác không nhận nhầm H1; action hợp lệ mới sau success không bị khóa bởi key toàn cục.

Invariant: với một identity D đã nhận H1, khi kết quả receipt chưa rõ, sendCount(D)=1; retry D chỉ gọi waitForReceipt(H1). Sau receipt thành công, D vẫn có sendCount(D)=1 dù Proposal ID chưa đọc được. Identity trang gồm account, wallet chain, deployment chain/ID/governor/treasury và payload đã build; calldata chứa requestId.

Constraints: (1) kiểm thử kiểm soát thời điểm bằng deferred/barrier, không dựa sleep, RPC ngoài hay thứ tự chạy; (2) ví ký trực tiếp, API chỉ đọc, không thêm backend write/secret; (3) mục tiêu tải VT-062 là 60 task đồng thời và p95 ≤2.000 ms, nhưng phép đo fake receipt chỉ được kết luận cho harness local. Bảo đảm chỉ trong một component còn mounted; reload, multi-tab, ví thật và thay deployment khi runtime nằm ngoài phạm vi đã chứng minh.

2. AI plan, review source và chiến lược rủi ro

Plan v1 được lưu trước patch tại spec-plan-v1.md; v2–v4 ghi điều chỉnh sau khi đọc source thật. **Điều chỉnh 1:** API ở indexer/api/server.ts là read-only, wallet-signed write đi từ frontend/src/chain/transactions.ts tới contract; BE03 dedup raw event và Treasury paidRequests không ngăn frontend gọi ví H2. Vì vậy chọn vertical slice trang tạo proposal + hook, dù phần việc trước

của sinh viên là TV2/backend. Khu vực frontend TV3 cần review phối hợp trước khi PR. **Điều chỉnh 2:** BudgetGovernor/OpenZeppelin tính proposal ID từ targets, values, calldatas, descriptionHash và từ chối exact payload đã đề xuất; hậu quả chắc chắn cần kiểm là lời gọi ví/giao dịch thừa, không khẳng định có hai ProposalCreated. **Điều chỉnh 3:** requestId nằm trong state của trang, không persist sau reload; thu hẹp AC trong cùng mount. **Điều chỉnh 4:** waitForReceipt phân biệt timeout với revert/cancel/repriced; chỉ timeout sau hash mới tiếp tục H1. Các nguồn và line review nằm ở plan-review-v2.md, plan-review-v4.md và evaluation-review.md.

Equivalence partitioning chọn đại diện theo trạng thái hash và receipt, vì đây là ranh giới đổi hành vi:

Partition	Đại diện	Oracle quan trọng
EP1 — trước hash	ví reject 4001	retry có thể gửi lại; không wait hash rỗng
EP2 — H1, wait lỗi tạm	H1 + RPC timeout	send 1; retry wait(H1)
EP3 — H1, wait pending	deferred receipt	overlap bị chặn, chưa báo success
EP4 — receipt success có ID	H1 + ID 42	callback 1, UI ID 42, không gửi lại
EP4b — receipt success thiếu ID	H1 + decoder null/throw	success/H1, handler chặn duplicate
EP5 — terminal	revert/cancel/repriced	không nhầm timeout với terminal; repriced giữ action
EP6 — identity mới	payload/account khác, H2	không gắn H1 vào yêu cầu khác

H1/H2, requestId và lỗi cố định là dữ liệu đại diện có oracle rõ; deferred Promise tạo overlap thay vì ngủ theo đồng hồ. Hai negative tests là từ chối ký và revert/cancel; failure path là timeout sau H1 và F1 decoder/refresh lỗi. UI/component test kiểm luồng submit; hook unit test kiểm state machine; API/PostgreSQL integration test chỉ kiểm mapper response. E2E extension wallet không cần để chứng minh RED local và chưa được chạy, nên không dùng tên E2E cho happy-dom.

3. Test cases và traceability

ID	Req/risk; level	Tiền điều kiện/ input	Expected	Ưu tiên
T01	AC1/3; unit	send→H1, receipt success	send 1, wait H1, callback 1	P1
T02	AC2; unit/UI	H1, wait timeout, retry cùng draft	send 1, wait H1 hai lần	P0
T03	EP3; unit/load	wait H1 còn pending, overlap	null bị chặn; không send thêm	P0
T04	AC4; unit negative	ví reject trước hash	retry được send, không wait trước đó	P1
T05	AC3; UI	success H1, ID 42	hiển thị ID; submit lại không send	P0
T06	F1; UI negative	success H1, decoder null	success/H1; handler send count 1	P0
T07	F1; UI failure	decoder throw, refresh reject	vẫn success/H1; submit trực tiếp không send	P0

ID	Req/risk; level	Tiền điều kiện/ input	Expected	Ưu tiên
T08	EP5; unit	receipt revert/ cancel	terminal, không callback success	P1
T09	EP5; unit	repriced	nhận receipt của cùng action	P1
T10	AC5; UI	đổi payload/ account khi H1 pending	không nhận nhầm H1; không send H2	P1
T11	AC5; UI	payload khác sau success	gửi H2, nhận đúng ID/hash mới	P1
T12	mutation; API/PG	requestId bytes32 khác proposalId decimal	list/detail map đúng requestId	P1
T13	NFR; load	60 duplicate + 60 distinct ×100 wave×3	đếm outcome/send, p95 riêng từng loại	P1

Traceability chi tiết ở traceability-green.md. T02/F1 đi qua CreateProposalPage.vt062.test.tsx và hook thật; T12 dùng tests/api/vt062-request-id.test.mjs với PostgreSQL 18.4/HTTP thật. T13 chạy frontend/scripts/vt062-load.load.tsx: hook production trong React thật, boundary ví/receipt giả. T03/T13 không coi null của lời gọi bị chặn là receipt thành công.

4. Test-first RED → patch → GREEN

RED ban đầu. Test-only commit 9a87741ae4728edb4092ece6d8d9e4c90f7650b7; hai assertion fail đúng hành vi: unit và form UI expected send count 1, actual 2 sau H1 + timeout. docs/evidence/vt062/red-targeted.txt và red-ui.txt ghi command, exit 1. Không phải import/package/selector lỗi. Sau khi sinh viên đối chiếu RED, patch 8a4cb6474775d210d6126b9faa3fab87b4bc2d07 giữ {hash,key} trong hook; retry cùng key wait H1, key khác bị chặn khi kết quả chưa rõ. GREEN: hai file RED 10/10, frontend suite 29/29 lúc đó, lint/build exit 0 (green-*). Đây là evidence lịch sử trước F1.

F1 RED và GREEN. Test-only 198fd2263040db52b13219daafbf38896ba0a6af cho receipt H1 success, decoder null, form/requestId giữ nguyên; lần submit sau gọi sendGovernor hai lần (expected 1) tại assertion test. f1-red-page-final.txt exit 1; F1 RED evidence commit 9e5458f. Sinh viên đã xác nhận nguyên nhân RED rồi yêu cầu patch. Commit 0d0a5f697f3ccc1a76333177f34508f721e8a2b4 lưu success/hash theo request key ở trang, kiểm map trong handler trước gọi run và báo "đã xác nhận, chưa đọc được Proposal ID" cùng H1. Không sửa hook dùng chung. Test cũ giữ expected; bổ sung submit event trực tiếp, decoder throw/refresh lỗi và payload mới sau success. Trên source F1 cuối: frontend 33/33, lint/build exit 0 (f1-green-final-*). Source SHA/diff ở f1-green-source-identity.txt, f1-green-code-tests.diff.

5. Mutation, coverage và flaky risks

Mutation M1 đúng biến thể: trong mapper indexer/api/server.ts:381, tạm đổi response requestId lấy row.proposal_id thay row.request_id. Fixture API có requestId bytes32 0xabab...abab, proposalId decimal khác hẳn; calldata, descriptionHash và proposalId được tính hợp lệ; migration 001–005, deployment, publication và snapshot mới mỗi lượt. **Good-pass → mutant-fail → restored-pass**: test list/detail đúng ở code tốt; mutant fail assertion requestId expected bytes32, actual proposalId decimal, không fail setup. Log/diff/source SHA ở docs/evidence/

vt062/mutation/ và evaluation-runbook.md. Mapper/API không đổi sau F1; kết quả mutation là evidence trước F1, không gọi là lượt mutation mới.

Coverage V8 **sau F1** chỉ include transactions.ts, useTransaction.ts, CreateProposalPage.tsx: tổng **75,13% line / 57,46% branch**; hook 100/96,55; trang 94,91/79,68; transactions 51,13/37,50 (f1-coverage-final-run.txt). Các nhánh thiếu gồm fallback account/deployment của trang, generic catch và helper cũ ngoài slice; decoder thật/guard chain trong UI bị mock. Tỷ lệ không thay oracle hành vi hay API mutation.

Flaky checks **sau F1**: 30 process UI ×5 test, seeds 62301–62330, 0 fail; 6 lượt suite shuffle ×33 test, seeds 62401–62406, 0 fail. Mock reset, React root unmount, deferred receipt và không sleep cố định; raw log ở f1-flaky-*-final-*. Rủi ro time/network thật, thứ tự trên hệ khác và concurrency ví/RPC vẫn chưa được loại trừ.

6. Thí nghiệm 60 task và giới hạn kết luận

Ba process độc lập; mỗi process cho từng workload 10 wave warm-up riêng, rồi 100 wave đo ×60 task. Harness/evidence commit ae7f6e9928326e6bcce32f121439259efe9a582a. Raw CSV gồm 39.600 mẫu (3.600 warm-up, **36.000 measured**), identity, hash, latency, outcome, error, send count và peak overlap. Clock monotonic performance.now() từ ngay trước useTransaction.run đến Promise settle; fake receipt mở ngay khi số waiter kỳ vọng vào barrier, không cài delay. P95 nearest-rank = phần tử ở index ceil(0,95×N)-1 sau sort; script tính lại từ CSV 6 chữ số thập phân.

Workload/ outcome measured	N	p50 ms	p95 ms	max ms	Fake send	Peak logical / waiter
DUPLICATE — receipt success đầu wave	300	0,089555	0,184630	0,609894	300 tổng workload	60 / 1
DUPLICATE — bị chặn khi in-flight	17.700	0,036675	0,096189	0,399151	0 từ task bị chặn	60 / 1
DUPLICATE — cached result	0	—	—	—	—	—
DISTINCT — receipt success	18.000	0,097580	0,262156	0,484102	18.000	60 / 60

unexpected_null, sai receipt và exception đều 0; 60 logical task thực sự overlap mỗi wave. DUPLICATE có 59 null bị chặn/wave, **không** có kết quả cached hay 60 success. Mỗi identity DISTINCT nhận đúng hash riêng; callback/send count khớp. P95 của các loại outcome đo được đều dưới 2.000 ms, nhưng đây **chỉ là độ trễ logic local của hook với fake send/ receipt**. Nó không đo F1 submit handler ở tải 60, extension wallet, RPC, indexer/API, chain finality hay SLA 60 người dùng thật. Nếu rubric đòi SLA end-to-end, tiêu chí đó **chưa được chứng minh**. Raw/summary/validation ở docs/evidence/vt062/load/; harness và script tóm tắt có trong repo.

7. Reflection — bản nháp AI hỗ trợ, cần sinh viên tự chỉnh (250–350 từ)

Việc làm bài theo quy trình test trước code cho thấy một assertion đúng hành vi có giá trị hơn một tỷ lệ coverage đẹp. Trường hợp đầu tiên bắt đầu từ hash H1 đã được ví trả về và một lần chờ receipt bị lỗi. Nếu chỉ nhìn thông báo lỗi, rất dễ xem lần bấm tiếp theo là một giao dịch mới. Test dùng deferred Promise đã chỉ ra đúng thời điểm state còn pending và chứng minh lời gọi gửi ví thứ hai. Sau đó, kiểm source Governor giúp sửa cách diễn đạt hậu quả: exact payload có thể bị từ chối ở contract, nhưng lời gọi ví thừa vẫn là lỗi cần xử lý.

Finding F1 là bài học thứ hai về oracle. Test success ban đầu luôn có Proposal ID nên không nhìn thấy trường hợp receipt đã xác nhận mà decoder trả null. Khi bổ sung điều kiện này, assertion gửi một lần lại fail. Patch sau F1 cần lưu kết quả thành công theo identity của request và kiểm ngay tại submit handler; trạng thái disabled của nút không đủ nếu sự kiện submit vẫn được gọi. Test lỗi refresh còn nhắc rằng đọc kết quả phụ trợ không được đảo trạng thái của transaction đã thành công.

AI giúp tạo plan, test, patch và log nhanh, nhưng các đầu ra đó vẫn cần đối chiếu với source, fixture và command thực chạy. Hai điểm cần giữ sự hoài nghi là mock decoder không chứng minh log chain thật và p95 rất nhỏ của fake receipt không chứng minh độ trễ hệ thống. Mutation với PostgreSQL thật kiểm được một mapper cụ thể, chứ không bảo vệ mọi lớp chống trùng. Trước khi nộp, sinh viên cần tự đọc diff, xác nhận phạm vi phối hợp frontend, sửa bản reflection theo trải nghiệm cá nhân và quyết định có cần thử ví/chain thật hay không. Báo cáo này ghi kết quả AI đã thực hiện; chưa có xác nhận sinh viên đã review hoặc chấp thuận patch.

PHỤ LỤC — ngoài giới hạn 8 trang nội dung chính

A. Evidence index và provenance

Nhóm	Artifact/nguồn
Baseline/plan	spec-plan-v1.md, plan-review-v2.md–v4.md, docs/evidence/vt062/checkout-and-environment.md
RED/GREEN ban đầu	red-targeted.txt, red-ui.txt, green-original-two.txt, green-frontend-suite.txt, green-provenance.md
F1	f1-red-review.md, f1-red-page-final.txt, f1-green-review.md, f1-green-final-suite.txt, f1-green-code-tests.diff
Mutation	mutation/good-02, mutation/mutant-01, mutation/restored-01, mutation/requestid-mutant.diff, evaluation-runbook.md
Coverage/flaky	f1-coverage-final-run.txt, f1-coverage-final/coverage-summary.json, f1-flaky-ui-30-final-*, f1-flaky-suite-6-final-*
Load	load/raw-run-1.csv–raw-run-3.csv, load/summary-run-*.json, load/aggregate.json, load/validation.txt

B. Lệnh và log đại diện

```
RED gốc: npm run test --prefix frontend -- src/pages/CreateProposalPage.vt062.test.tsx
  expected sendGovernor = 1, actual = 2; exit 1 (red-ui.txt)
F1 RED: npm run test --prefix frontend -- src/pages/CreateProposalPage.vt062.test.tsx -t 'does not submit the same draft again'
  expected sendGovernor = 1, actual = 2; exit 1 (f1-red-page-final.txt)
F1 GREEN: npm run test --prefix frontend
  5 files, 33 tests passed; exit 0 (f1-green-final-suite.txt)
MUTANT: API/PostgreSQL mới, list.requestId actual proposalId decimal, expected bytes32; exit 1
LOAD: 3 lượt, mỗi lượt 100 measured waves/workload ×60 task; 3/3 exit 0
```

Log đầy đủ kèm exit code ở thư mục evidence nêu trên. Ảnh/screenshot ví thật không có vì không chạy extension wallet; không dùng UI happy-dom làm ảnh E2E blockchain.

C. AI Usage Log đầy đủ

Nội dung dưới đây được đồng bộ từ AI-usage-log.md khi xuất PDF. Nó ghi prompt/output đã dùng và phân biệt phần AI với xác nhận của sinh viên.

AI Usage Log — BT3 / VT-062

Sinh viên: Đào Ngọc Cường — 23001841 · Team SE2026-T16
Ngày: 2026-10-30 UTC · Trạng thái: ghi nhận hoạt động AI trong lượt khảo sát/spec/test RED; chưa có review/chấp thuận của sinh viên.

Prompt thực dùng

Nguồn prompt là yêu cầu của sinh viên trong cuộc hội thoại này. Các phần trực tiếp dùng cho phạm vi:

> “Cho phép người dùng gửi lại cùng một yêu cầu mà không tạo dữ liệu hoặc giao dịch trùng.”
>
> “Risk trọng tâm: test phụ thuộc thời gian, network hoặc thứ tự chạy nên bị flaky.”

> “Kỹ thuật bắt buộc: equivalence partitioning.”
>
> “AI viết test trước implementation; tôi phải kiểm tra test fail đúng lý do trước khi sửa code.”
>
> “Nếu chọn hướng frontend, test trọng tâm: lần gửi đầu trả hash H1; lần chờ receipt đầu thất bại có kiểm soát; người dùng thử lại qua luồng sản phẩm đang có; expected không gọi gửi thêm giao dịch, tiếp tục theo dõi H1 và trả đúng kết quả.”
>
> “Lưu plan ban đầu rồi review bằng repository thật. Sửa hoặc bác bỏ ít nhất hai điểm có evidence cụ thể.”
>
> “DỪNG TẠI ĐÂY. Chờ tôi xác nhận đã kiểm tra nguyên nhân RED trước khi sửa production code.”

Output AI đã dùng

1. Đọc source thật và chọn slice frontend sau khi xác nhận hook bỏ hash ở lần gọi tiếp theo và trang mở nút khi phase error. Chọn TV3 dù vai trò cũ là TV2 vì API/indexer/contracts không thể ngăn frontend ký/gửi transaction mới sau H1; ghi rõ giới hạn lớp.
2. Tạo spec/plan v1 và review v2 với các phân hoạch EP1–EP6, 12 case tối thiểu, ranh giới guarantee và kế hoạch mutation response requestId.
3. Tạo frontend/src/chain/useTransaction.vt062.test.ts trước mọi production patch. Test dùng hash/receipt fixture cố định, waitForReceipt mock, deferred Promise và reset state mỗi test; không dùng RPC, timer ngủ hoặc DB.
4. Chạy test trên baseline ae8c3533e14a01f74ba4ea7b00c74365e1651be1. Test retry thất bại ở assertion sendCount=1; actual sendCount=2. Hai test control pass. Production source không đổi.
5. Lưu command/output trong docs/evidence/vt062/. Đã cài đúng Node 24.15.0 và frontend dependencies theo lockfile trong clone cô lập; không thêm dependency.

Hạn chế AI được nhận diện trước khi review của sinh viên

- Unit harness mô phỏng React hook state và chứng minh nhánh logic của hook; nó không render CreateProposalPage trong browser và không tự chứng minh nút UI/extension wallet.
- API requestId mapping test là yêu cầu mutation bổ trợ, không phải oracle cho việc ví có gửi thêm transaction hay không.
- Hiệu năng 60 task dùng fake receipt chỉ là benchmark logic ứng dụng. Không được trình bày là độ trễ RPC, chain, API production hoặc SLA thật.

Chưa ghi reflection cá nhân thay sinh viên. Chưa ghi rằng sinh viên đã đọc source, xác nhận nguyên nhân RED hoặc chấp thuận patch.

Bổ sung sau audit đường kiểm thử UI

Sau khi thấy unit harness hook chưa render trang/nút, AI đã tạo plan-review-v3.md và bổ sung một test trang CreateProposalPage. Vì package ban đầu không có DOM emulator, AI đề xuất duy nhất happy-dom 21.0.4; sau network approval, dependency được cài theo lockfile. UI test dùng MemoryRouter, mock API/wallet/transaction và deferred receipt Promise; không có RPC/chain thật. Nó submit form, gây lỗi RPC timeout có kiểm soát rồi bấm lại nút đang bật. Oracle fail là sendGovernor expected 1, actual 2. 16 test khác pass trong full frontend suite; thêm unit hook test thứ hai cũng fail tại send count.

Ghi chú dependency ban đầu “không thêm dependency” áp dụng cho unit harness đầu tiên; plan-review-v3 sửa quyết định đó để đáp ứng yêu cầu test qua luồng sản phẩm.

Correction sau khi đọc OpenZeppelin Governor cùng revision

AI ban đầu mô tả khả năng indexer ghi thêm proposal cho exact same payload. Kiểm tra submodule source cùng commit cho thấy Governor tính lại cùng proposalId và revert lần propose trùng trước ProposalCreated. Log này sửa kết luận: test RED chứng minh transaction send thứ hai, không chứng minh proposal row thứ hai. Nếu payload đổi nhưng reuse requestId, proposalId có thể khác; đó là giới hạn đã ghi trong plan-review-v4.md.

Lượt patch → GREEN sau xác nhận RED của sinh viên

Prompt thực dùng: Sinh viên xác nhận: “Tôi xác nhận log RED thất bại đúng tại assertion: expect(uiHarness.sendGovernor).toHaveBeenCalledTimes(1) Actual: 2 lần gọi.” Sinh viên yêu cầu tiếp tục patch → GREEN, chuyển clone sang ~/projects, chốt commit test-only trước production, giữ identity của draft/payload/account/chain/deployment, xử lý timeout/revert/cancel, chạy lại hai test RED/suite/lint/build, lưu evidence và không làm mutation/tải/PDF/push/PR/merge trong lượt này.

Output AI và thao tác thực dùng:

1. Chuyển clone bằng mv sang /home/cuongdz/projects/dao-budget-vt062-individual khi đường đích chưa tồn tại. Lệnh chuyển thành công nhưng git status gắn sau lệnh tại working directory /tmp trả exit 128 vì /tmp không phải repository; kiểm tra lại tại đường đích xác nhận clone nguyên vẹn.
2. Chốt commit 9a87741ae4728edb4092ece6d8d9e4c90f7650b7 chỉ gồm hai test RED và happy-dom trong package/lock. Git chưa cấu hình author; commit đầu bị từ chối, sau đó dùng Codex <codex@users.noreply.github.com> cho riêng lệnh commit. Không mạo nhận sinh viên viết test AI.
3. Sửa useTransaction.ts giữ hash + key sau khi ví trả H1; cùng key retry wait(H1), key khác của trang proposal bị chặn. CreateProposalPage.tsx tạo key từ account, chain, deployment và exact payload. transactions.ts phân loại receipt revert/cancel/replace là terminal, timeout vẫn retryable. Callback của caller không truyền key giữ hành vi mới action như trước.
4. Giữ nguyên hai assertion RED. Bổ sung tests cho identity đổi payload/account, pending concurrency, terminal outcomes, repriced và tác động hook dùng chung. Một lỗi trong test mới (beforeEach vô tình trả mock function) đã được AI phát hiện khi chạy và sửa. Build đầu exit 2 vì parameter property không tương thích erasableSyntaxOnly; đổi thành field tường minh và build qua.
5. Chạy lại: hai file RED gốc 10/10 test qua, frontend suite 29/29 qua, lint exit 0 (8 warning wallet.tsx sẵn có), build exit 0. Lưu output và exit code ở docs/evidence/vt062/green-*.txt; source commit 8a4cb6474775d210d6126b9faa3fab87b4bc2d07, diff ở green-code-and-tests.diff. Checkout gốc được đối chiếu status/HEAD không đổi.

Độ tin cậy cần người review: JSON key cần được đối chiếu với payload thật và lựa chọn ví khi thử tích hợp; tests hiện dùng mock ví/RPC. Phân loại replacement dựa trên callback viem được unit-test bằng fake client; chưa xác nhận bằng extension wallet hoặc Anvil. Sinh viên đã xác nhận nguyên nhân RED theo log; chưa có tuyên bố sinh viên đã review/hiểu/chấp thuận patch GREEN.

Mutation response field, tải 60 task, coverage, flaky experiments và PDF chưa thực hiện theo giới hạn lượt này. Không viết reflection cá nhân thay sinh viên.

Lượt review diff → mutation → coverage → flaky checks

Prompt thực dùng: Sinh viên yêu cầu tiếp tục tại clone cá nhân, kiểm branch/HEAD/status và review production/test/spec, tạo test API/PostgreSQL thật cho mutation requestId từ proposal_id thay vì request_id, chứng minh good-pass/mutant-fail/restored-pass trên DB/snapshot mới, đo line/branch coverage, kiểm ít nhất hai flaky risks bằng lặp/shuffle, lưu evidence và không push/PR/merge/tải 60/PDF. Sinh viên nhắc rõ “Không ghi rằng tôi đã review/chấp thuận patch.” Tin nhắn tiếp theo “countinue” được hiểu là tiếp tục công việc đang làm, không đổi phạm vi.

Output AI đã dùng và kết quả thực:

1. Xác minh đầu lượt HEAD b7528d3, branch cá nhân và status sạch; đọc diff production, tests, spec/plan v1–v4 cùng mapper indexer/api/server.ts:381. Ghi review ở evaluation-review.md; phát hiện finding có điều kiện F1 về receipt success thiếu Proposal ID và nút có thể mở lại. Chưa sửa production hoặc tuyên bố sinh viên review patch.
2. Viết test riêng tests/api/vt062-request-id.test.mjs và runner scripts/check_vt062_api_native.mjs. Fixture dùng requestId bytes32 cố định khác proposalId decimal, calldata disburse/descriptionHash/proposalId hợp lệ; migration 001–005, deployment, raw event, checkpoint và publication; HTTP server + PostgreSQL 18.4 thật. Test assert list/detail chính xác từ fixture và snapshot count 0 trước request. Commit test 4c5ab03 do Codex tạo. Root npm ci --offline exit 0.
3. Good code pass, mutant worktree thay đúng một expression ở mapper và fail tại assertion list.requestId (actual proposalId decimal, expected bytes32), restored code pass với cluster mới. SHA

mapper, diff, logs và cleanup ở docs/evidence/vt062/mutation/. Mutation được áp ngược rồi tháo worktree; không có mutation trong branch nộp.

4. Cài devDependency @vitest/coverage-v8@4.1.12 đúng phiên bản Vitest đã khóa; thêm test cancel khi RPC reject sau replacement và assertion UI không submit thêm sau success có Proposal ID. Commit 6144f1b do Codex tạo. Coverage V8 trên ba module production sửa: tổng line 72.94%, branch 51.74%; test suite 30/30 pass. Review chỉ ra nhánh guard/success thiếu event chưa được test dù tỷ lệ hook cao.
5. Thử flaky trên test cuối: 30 lượt logic ×16 test, 6 seeds shuffle ×30 test và ba file VT-062 chạy riêng; tất cả exit 0, raw outcomes lưu riêng. Test dùng deferred/barrier, mock reset, component unmount; API runner đóng server/pool/cluster. Không dùng retry để biến failure thành pass. Frontend lint/build exit 0 với warning baseline được ghi lại.

Hai điểm AI-generated vẫn cần kiểm thêm: (a) UI mock proposalIdFromReceipt luôn trả 42 ở đường bình thường nên có thể che receipt success thiếu event; F1 đã ghi. (b) Phép đo coverage chỉ include ba frontend module; API mapper được kiểm bằng mutation/PostgreSQL nhưng không có line/branch coverage từ cùng provider. E2E ví/Anvil và tải 60 tác vụ chưa chạy. Chưa ghi reflection cá nhân thay sinh viên và chưa nói sinh viên đã review/chấp thuận patch.

Lượt tái hiện F1, test-first RED

Prompt thực dùng: Sinh viên yêu cầu “chỉ tái hiện finding receipt thành công nhưng không giải mã được ProposalCreated/Proposal ID”, kiểm HEAD/branch/status, viết regression test đi qua CreateProposalPage và hook thật, fault inject bước lấy ID trả null, giữ nguyên draft/requestId, chạy test trên production code hiện tại, lưu expected/actual/SHA/diff và dừng nếu RED. Sinh viên yêu cầu rõ chưa sửa production, chưa tải/PDF/push/PR/merge.

Output AI thực dùng: Xác minh HEAD đầu lượt a997bf8 và status sạch; đọc evaluation-review, trang, hook, decoder. Thêm một test UI vào CreateProposalPage.vt062.test.tsx trước mọi production patch. Test dùng deferred Promise để điều khiển receipt success H1, mock proposalIdFromReceipt trả null cho logs rỗng; API/wallet/send được mock, trang và hook là implementation thật. Test assert UI success/H1, không có Proposal ID/revert, form/requestId không đổi; click lại qua nút đang có rồi expected sendGovernor count 1. Chạy riêng case, cả file, full suite: đều fail ở send count expected 1 actual 2; 2 UI case cũ và 30 test suite khác pass. Build/lint exit 0. Lưu log và diff ở docs/evidence/vt062/f1-*; production source SHA không đổi, production diff rỗng. Chốt commit test-only 198fd2263040db52b13219daafbf38896ba0a6af bằng author Codex. Chưa sửa production hoặc tuyên bố sinh viên đã review/chấp thuận F1.

Giới hạn: Receipt success với logs: [] và decoder trả null là fault injection; test chứng minh hành vi component/hook dưới điều kiện này, không chứng minh tần suất của điều kiện với ví/chain thật. Người học cần kiểm nguyên nhân RED trước khi AI thực hiện patch F1. Chưa ghi reflection cá nhân.

Lượt patch F1 → GREEN theo xác nhận RED

Prompt thực dùng: Sinh viên xác nhận RED “receipt H1 thành công, decoder trả null, người dùng submit lại cùng draft/requestId và sendGovernor được gọi hai lần thay vì một”, rồi yêu cầu patch nhỏ chỉ cho F1, giữ success/hash/identity, guard ở submit handler, thông báo khi thiếu Proposal ID, không gửi khi refresh, giữ binding payload/account/chain/deployment và hành vi caller khác. Yêu cầu giữ assertions RED, chạy frontend suite/lint/build, coverage/flaky lại trên source mới, lưu log/SHA/diff, cập nhật traceability, commit local; chưa tải 60/PDF/push/PR/merge và không nói sinh viên đã review patch.

Output AI thực dùng: Đầu lượt xác minh nhánh individual/23001841-vt062-idempotent-retry, HEAD 9e5458f và status sạch; đọc f1-red-review.md, F1 test, trang, hook, decoder và AGENTS.md. Chỉ sửa CreateProposalPage.tsx: lưu receipt success {hash, proposalId} theo requestKey đã có; guard cùng key trong submit handler trước transaction.run và trước assertWritable cho write mới, để refresh API lỗi không đảo success; catch lỗi đọc ID rồi vẫn giữ H1; hiển thị thông báo xác nhận/H1 khi thiếu ID. useTransaction.ts và transactions.ts không sửa. Giữ nguyên F1 assertion send count 1 và hai RED gốc. Bổ sung test submit event trực tiếp (bỏ qua disabled), decoder throw + auto/manual refresh lỗi, và payload khác sau H1 success.

Sau patch đầu, targeted UI 5/5, hai file 13/13, suite 33/33, lint/build exit 0. AI review thấy guard nên chạy trước assertWritable khi summary refresh thất bại; chỉnh thứ tự và thêm oracle không xuất hiện lỗi write sai trong test. Các log đầu giữ riêng tên không có final. Trên source cuối commit 0d0a5f697f3ccc1a76333177f34508f721e8a2b4, hai file targeted 13/13, suite 33/33, lint/build exit 0. V8 ba module: line 75.13%, branch 57.46%; trang 94.91/79.68. Flaky checks cuối: 30 lượt UI ×5 test (seeds 62301–62330) và 6 lượt full suite ×33 test shuffle (seeds 62401–62406), tất cả exit 0. Command, raw log, SHA/diff và giới hạn ở docs/evidence/vt062/f1-green-review.md. API mapper SHA không đổi; mutation PostgreSQL trước F1 là evidence lịch sử, không lặp phép đo. Chưa chạy tải/PDF hoặc ghi reflection thay sinh viên. Sinh viên chưa review/chấp thuận patch.

Lượt tải 60 task và đóng gói báo cáo

Prompt thực dùng: Sinh viên yêu cầu tiếp tục ở clone/nhánh cá nhân sau HEAD d7ecc1c, hoàn thành hai workload DUPLICATE và DISTINCT, mỗi workload 60 task chồng nhau, 10 warm-up +100 measured waves ×3 lượt độc lập, đo latency clock monotonic và nearest-rank p95, lưu raw sample/outcome/send/concurrency; không dùng fake result để nhận SLA thật. Sau đó viết báo cáo Markdown, PDF nội dung chính tối đa 8 trang, README chạy lại, Draft PR local và commit; giữ provenance, không push/PR/merge và không nói sinh viên đã review/chấp thuận patch.

Output AI thực dùng: Xác minh branch individual/23001841-vt062-idempotent-retry, HEAD d7ecc1c, status sạch và sự tồn tại của các commit baseline/test/patch/F1. Đọc plan v1–v4, traceability, AI log, RED/GREEN, mutation, coverage/flaky F1. Tạo frontend/scripts/vt062-load.load.tsx và config riêng: render hook useTransaction production trong React thật; fake send/waitForReceipt tại boundary, dùng barrier/deferred, không viết map dedup trong harness. Smoke 1 warm-up +2 measured waves/run 99 exit 0 ở /tmp. Ba process full run đều exit 0; raw 39.600 mẫu (36.000 measured), validation script kiểm đủ identity/hash/outcome/send và tính lại p95 nearest-rank. DUPLICATE measured 300 receipt success đầu wave, 17.700 blocked null, fake send 300; DISTINCT 18.000 receipt success/hash đúng, fake send 18.000; lỗi 0, peak logical 60 mỗi wave, peak waiter 1/60. P95 riêng lần mới/blocked/distinct là 0,184630/0,096189/0,262156 ms. Đây chỉ là logic local với fake ví/receipt; không phải SLA 60 người dùng thật, không đo F1 handler dưới tải. Chốt harness/evidence commit ae7f6e9928326e6bcce32f121439259efe9a582a; command, raw CSV, JSON và SHA ở docs/evidence/vt062/load/, load-runbook.md.

AI tạo BT3_23001841_VT-062.md, README, Draft PR, diff summary và renderer PDF. PDF dùng Markdown/WeasyPrint/PyPDF/PyMuPDF cùng DejaVu Sans, dependency Python cài tạm ngoài repo; kết quả 5 trang chính +5 trang phụ lục, reflection 344 từ, trích text tiếng Việt đủ, font DejaVu embedded, 0 clipped span và mọi trang render được (pdf-validation.txt, pdf-render-command.txt). Sau khi thêm harness/tài liệu, frontend suite 33/33, lint/build exit 0; log ở package-frontend-*. Reflection là bản nháp AI hỗ trợ, chưa phải lời xác nhận trải nghiệm của sinh viên. Sinh viên cần tự review diff, chỉnh reflection và quyết định khi nào publish nhánh. Không chạy tải chain/RPC thật, không push/PR/merge.